

Ordering Test Class Execution: Introduction

JUnit normally does **not** guarantee class execution order.

Sometimes order matters (e.g., integration tests that depend on state).

RP

by **Rodrigo Martins
Pagliares**

Last update: 12/23/2025



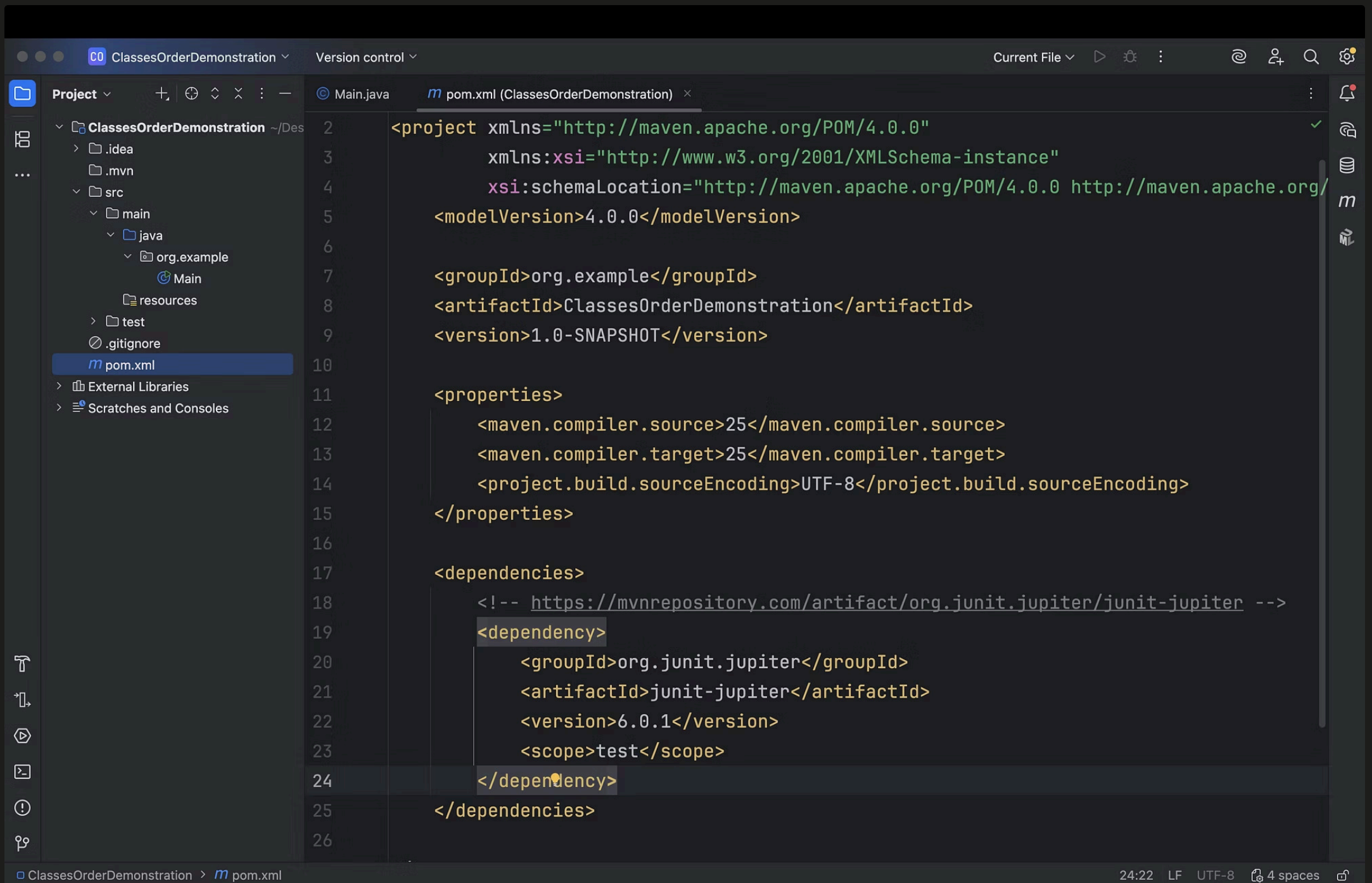
Ordering Test Class Execution: Introduction

- JUnit normally does **not** guarantee class execution order.
- Sometimes order matters (e.g., **integration tests** that **depend** on **state**).
- Example project in this lesson includes three classes:
 - UserServiceTest
 - ProductServiceTest
 - OrderServiceTest
- Each class has a `setup` method that prints a message when executed.



Create a new Maven Project with JUnit 6 support

Alternatively you can use **Gradle**.

A screenshot of an IDE window showing a Maven project named 'ClassesOrderDemonstration'. The left sidebar displays the project structure with folders like .idea, .mvn, src, main, java, org.example, Main, resources, test, .gitignore, and pom.xml. The main editor area shows the pom.xml file with the following content:

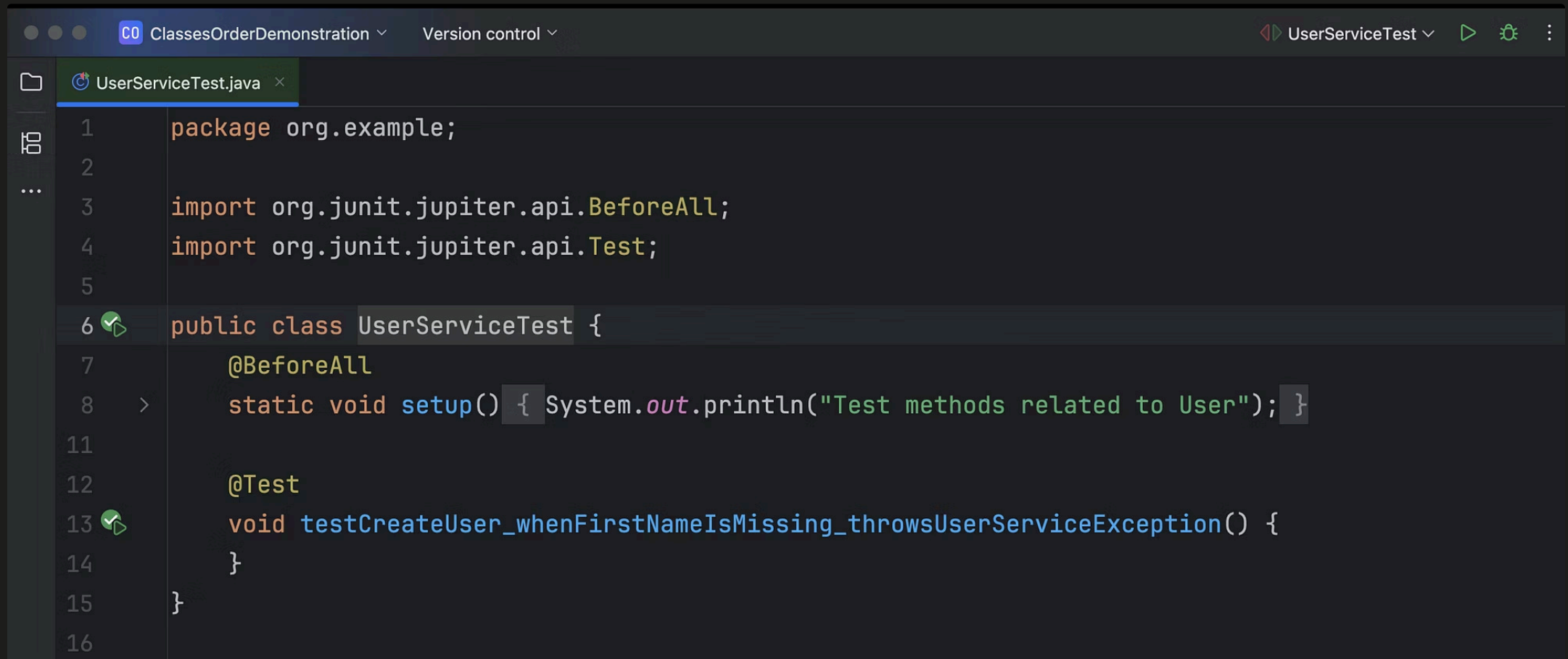
```
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/
5       <modelVersion>4.0.0</modelVersion>
6
7       <groupId>org.example</groupId>
8       <artifactId>ClassesOrderDemonstration</artifactId>
9       <version>1.0-SNAPSHOT</version>
10
11       <properties>
12         <maven.compiler.source>25</maven.compiler.source>
13         <maven.compiler.target>25</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15       </properties>
16
17       <dependencies>
18         <!-- https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter -->
19         <dependency>
20           <groupId>org.junit.jupiter</groupId>
21           <artifactId>junit-jupiter</artifactId>
22           <version>6.0.1</version>
23           <scope>test</scope>
24         </dependency>
25       </dependencies>
26
```

The status bar at the bottom shows 'ClassesOrderDemonstration > pom.xml' and '24:22 LF UTF-8 4 spaces'.

Create the UserServiceTest

UserServiceTest is a test class.

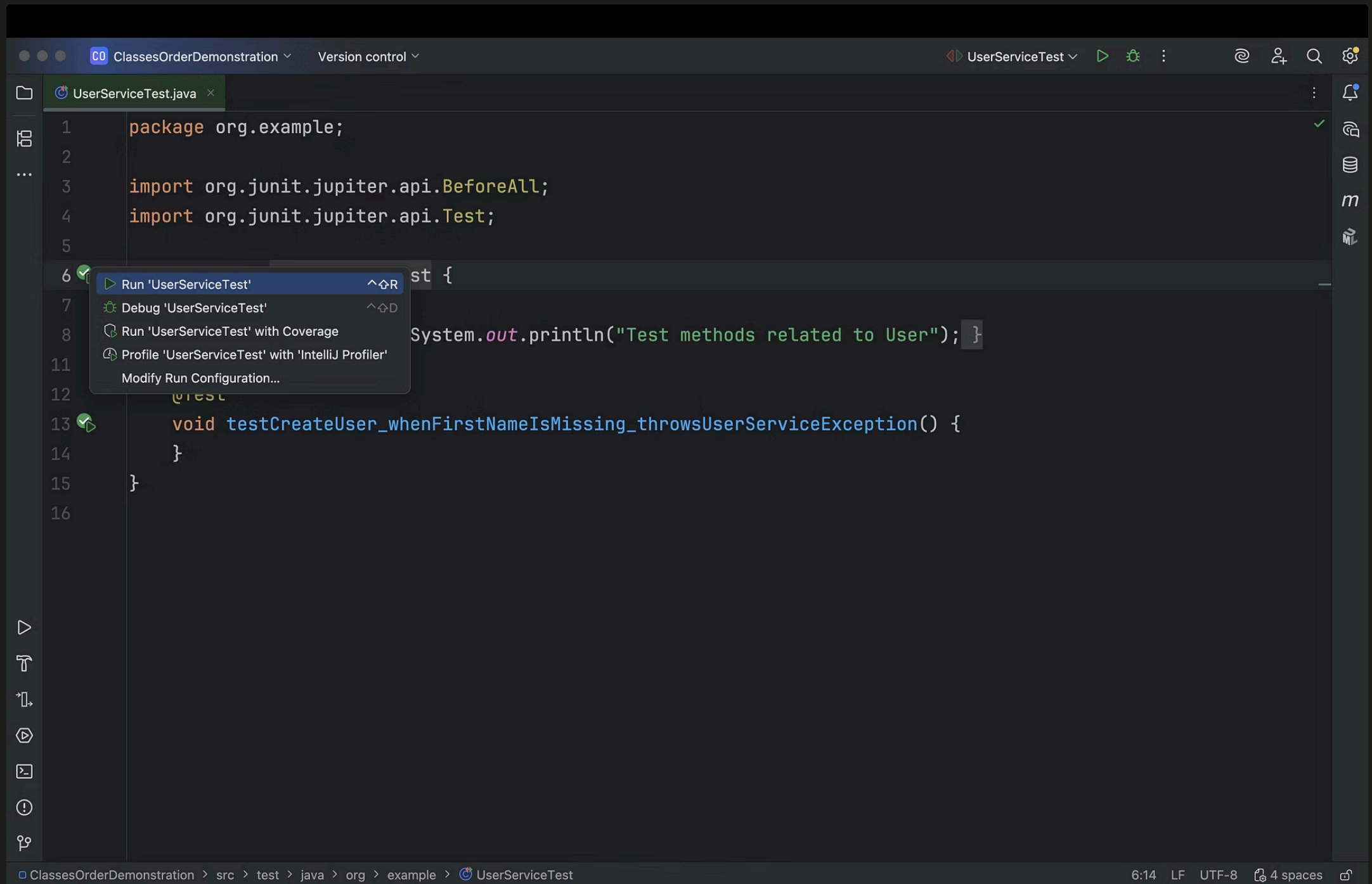
I created it in the package `org.example`: `src/test/java/org/example/UserServiceTest.java`

A screenshot of an IDE window titled 'ClassesOrderDemonstration' with a 'Version control' dropdown. The active file is 'UserServiceTest.java'. The code is as follows:

```
1 package org.example;
2
3 import org.junit.jupiter.api.BeforeAll;
4 import org.junit.jupiter.api.Test;
5
6 public class UserServiceTest {
7     @BeforeAll
8     > static void setup() { System.out.println("Test methods related to User"); }
9
10
11
12     @Test
13     > void testCreateUser_whenFirstNameIsMissing_throwsUserServiceException() {
14     }
15 }
16
```

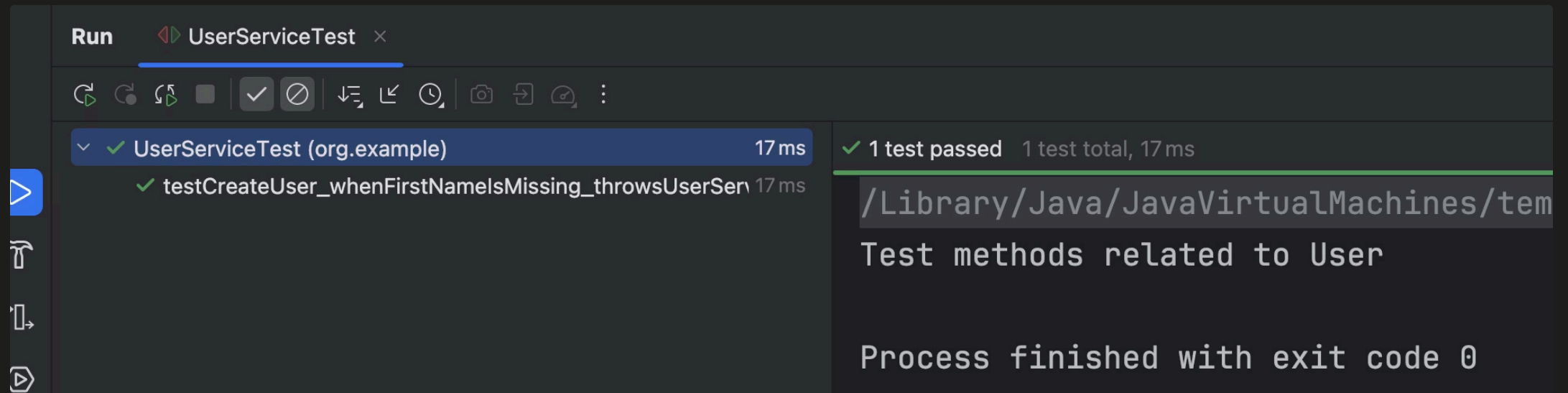
Run the Test Methods

Let's test our implementation so far...



Tests Passed!

The output "Test methods related to the User confirms our setup method is working (@BeforeAll)



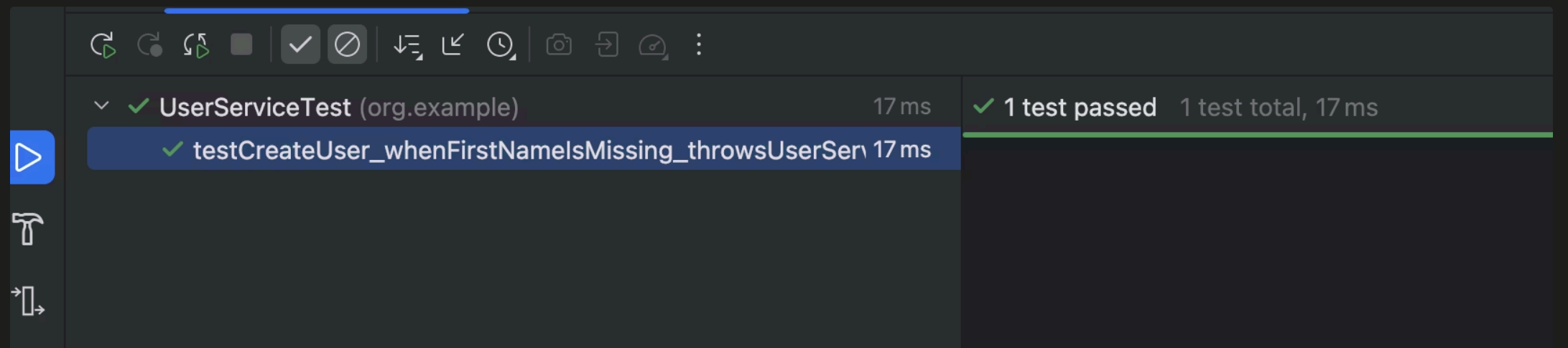
The screenshot shows the 'Run' window of an IDE. The title bar says 'Run' and 'UserServiceTest'. Below the title bar is a toolbar with icons for running, debugging, and other actions. The main area shows a tree view of the test results. The root node is 'UserServiceTest (org.example)' with a green checkmark and '17 ms'. It has a single child node 'testCreateUser_whenFirstNamelsMissing_throwsUserSer' with a green checkmark and '17 ms'. To the right of the tree view, a summary bar shows '1 test passed' and '1 test total, 17 ms'. Below this, the output text is displayed: '/Library/Java/JavaVirtualMachines/tem' and 'Test methods related to User'. At the bottom, it says 'Process finished with exit code 0'.

Test Method	Duration
UserServiceTest (org.example)	17 ms
testCreateUser_whenFirstNamelsMissing_throwsUserSer	17 ms

Summary: 1 test passed, 1 test total, 17 ms

Output: /Library/Java/JavaVirtualMachines/tem
Test methods related to User
Process finished with exit code 0

The green bar confirmed that our second test method passed, even though it is empty at this moment (@Test)



This screenshot is similar to the one above, showing the 'Run' window for 'UserServiceTest'. The tree view shows the same test results. The summary bar shows '1 test passed' and '1 test total, 17 ms'. The output text is not visible in this screenshot.

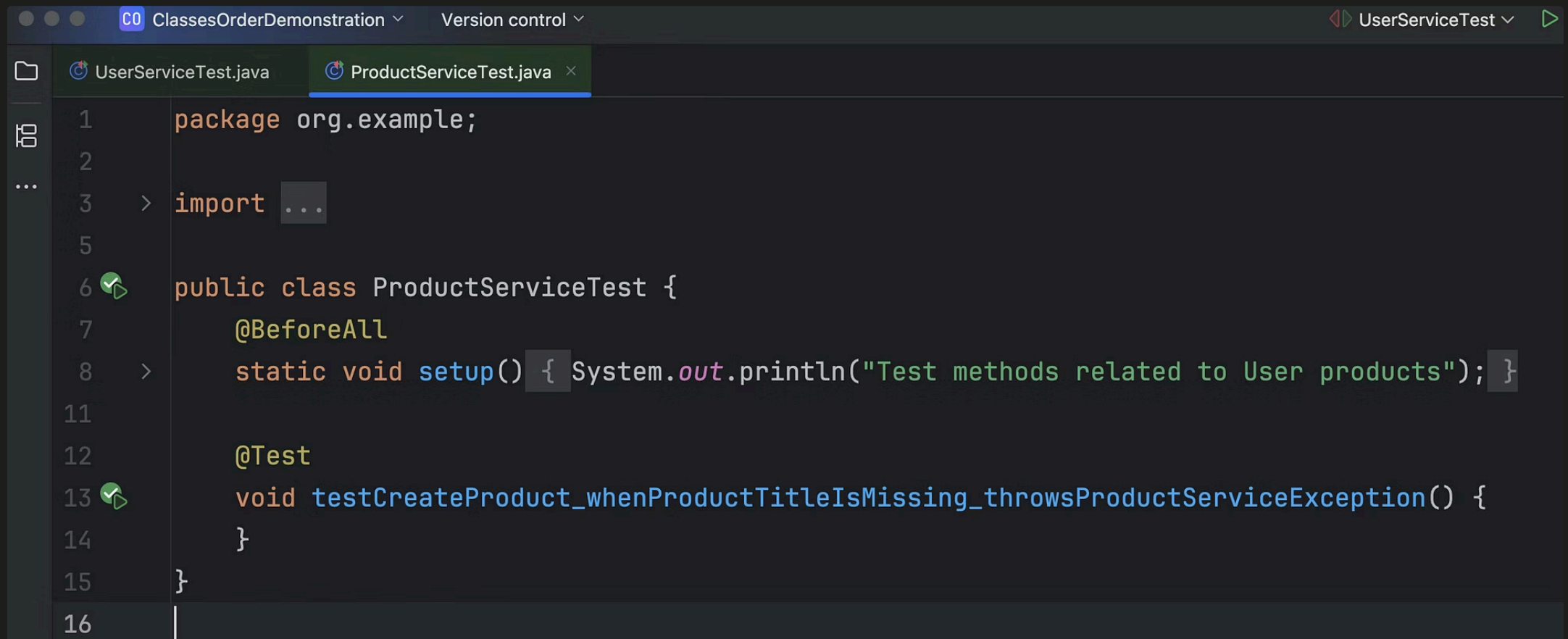
Test Method	Duration
UserServiceTest (org.example)	17 ms
testCreateUser_whenFirstNamelsMissing_throwsUserSer	17 ms

Summary: 1 test passed, 1 test total, 17 ms

Create the ProductServiceTest

ProductServiceTest is a test class.

I created it in the package `org.example`: `src/test/java/org/example/ProductServiceTest.java`

A screenshot of an IDE window showing the code for ProductServiceTest.java. The window has a title bar with "C0 ClassesOrderDemonstration", "Version control", and "UserServiceTest". The editor shows the following code:

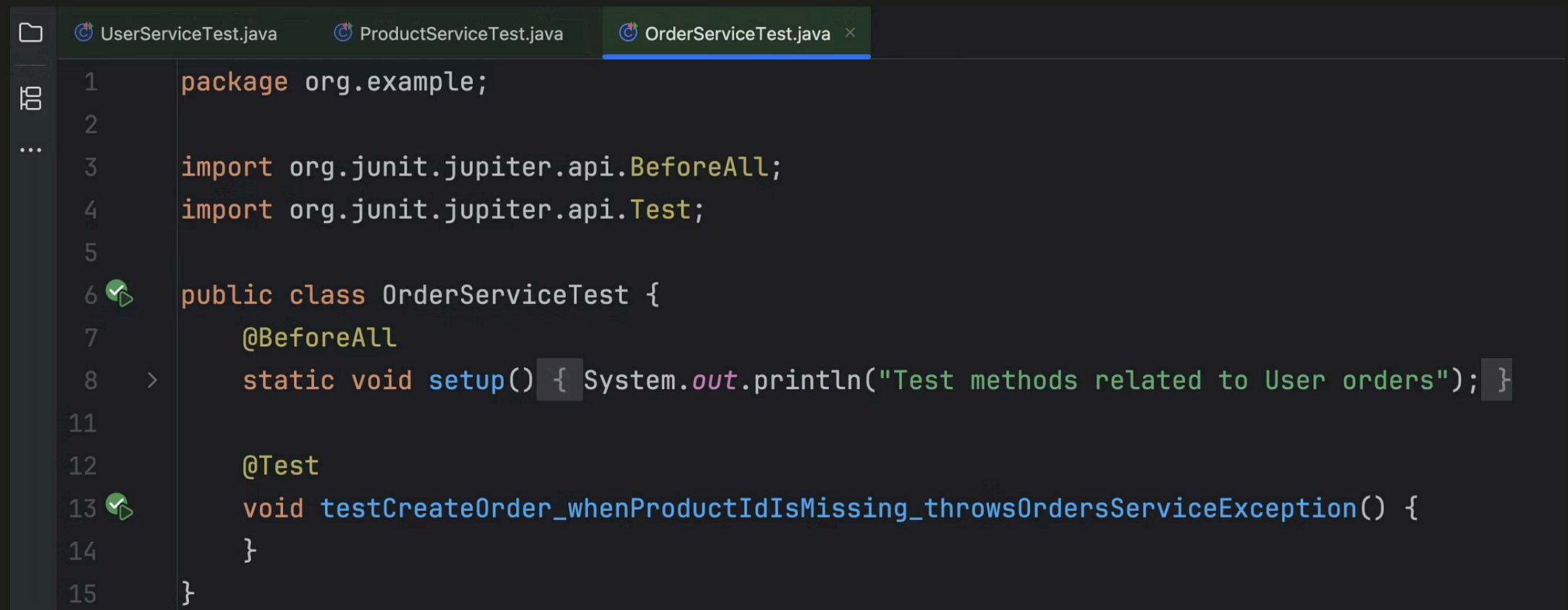
```
1 package org.example;
2
3 > import ...
4
5
6 public class ProductServiceTest {
7     @BeforeAll
8     > static void setup() { System.out.println("Test methods related to User products"); }
9
10
11
12     @Test
13     void testCreateProduct_whenProductTitleIsMissing_throwsProductServiceException() {
14     }
15 }
16
```

Run this class's test methods to confirm they are working properly.

Create the OrderServiceTest

OrderServiceTest is a test class.

I created it in the package `org.example`: `src/test/java/org/example/OrderServiceTest.java`

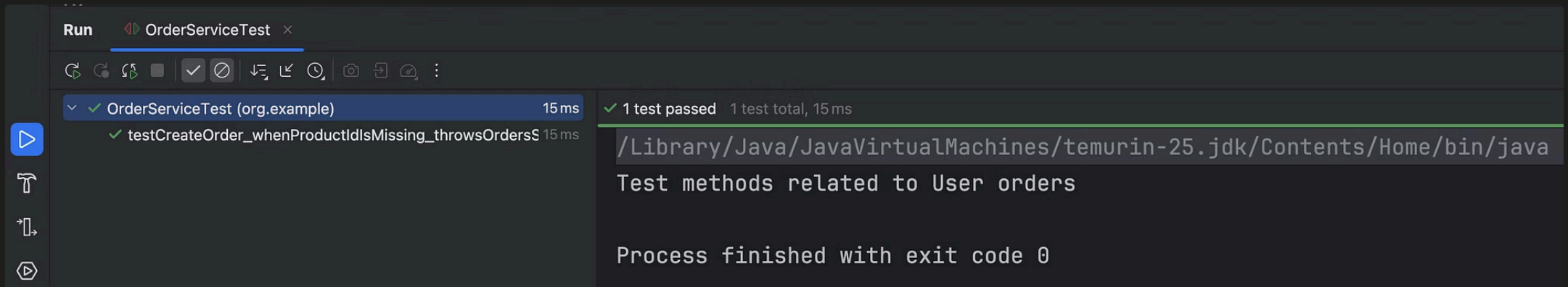


```
UserServiceTest.java  ProductServiceTest.java  OrderServiceTest.java x
1  package org.example;
2
3  import org.junit.jupiter.api.BeforeAll;
4  import org.junit.jupiter.api.Test;
5
6  public class OrderServiceTest {
7      @BeforeAll
8      > static void setup() { System.out.println("Test methods related to User orders"); }
11
12      @Test
13  void testCreateOrder_whenProductIdIsMissing_throwsOrdersServiceException() {
14      }
15  }
```

Run this class's test methods to confirm they are working properly.

Tests Passed!

The output "Test methods related to User orders confirms our setup method is working (@BeforeAll)



The screenshot shows the 'Run' window of an IDE. The title bar indicates the test class is 'OrderServiceTest'. The toolbar contains icons for running, debugging, and other test-related actions. The test results are displayed in a tree view on the left and a console view on the right. The tree view shows a single test method, 'testCreateOrder_whenProductIdsMissing_throwsOrdersS', which passed in 15 ms. The console view shows the output of the test, which is 'Test methods related to User orders', and the process finished with exit code 0.

Run OrderServiceTest x

✓ OrderServiceTest (org.example) 15 ms

✓ testCreateOrder_whenProductIdsMissing_throwsOrdersS 15 ms

✓ 1 test passed 1 test total, 15 ms

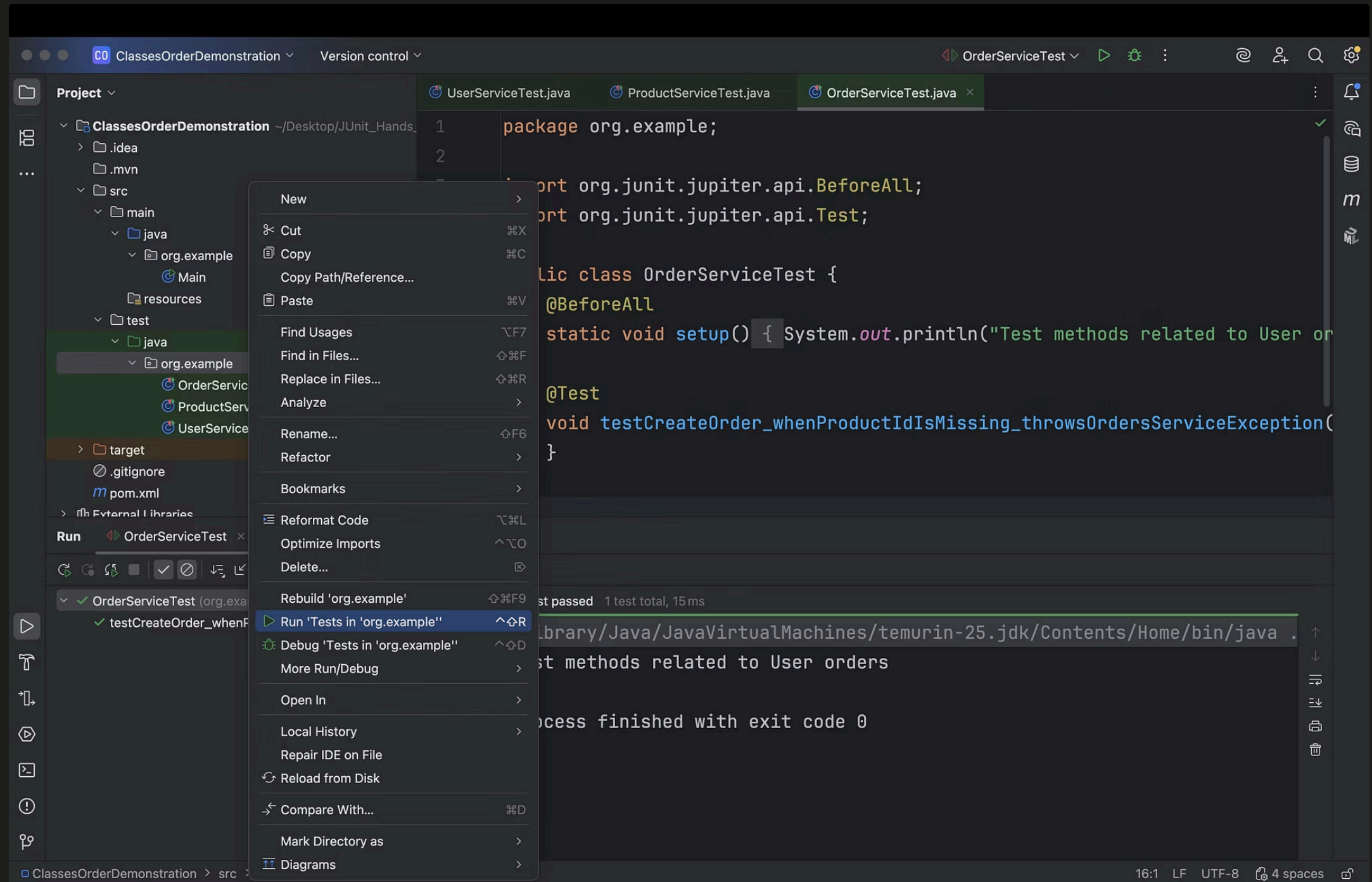
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java

Test methods related to User orders

Process finished with exit code 0

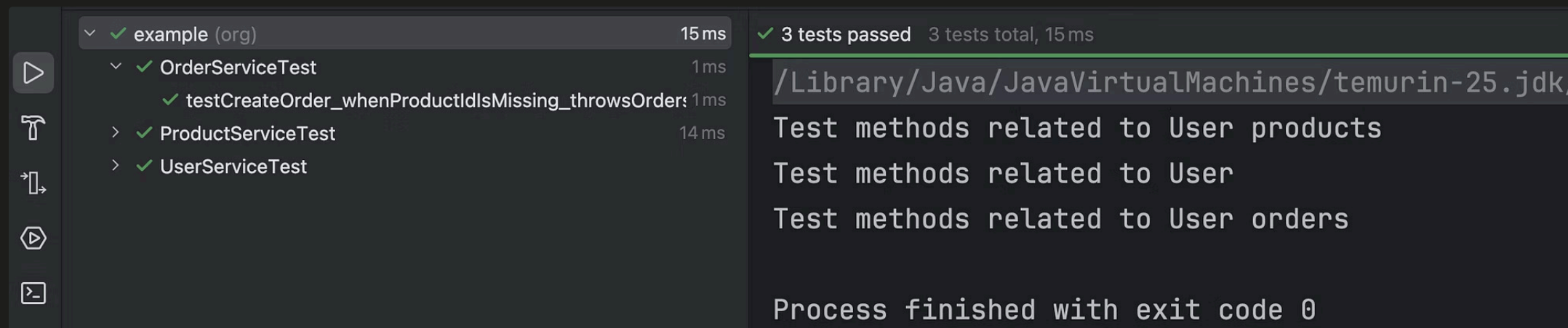
Run all 3 test classes at once

No order guaranteed!



Run all 3 test classes at once (contd.)

In this example, `OrderServiceTest` run first.



The screenshot shows an IDE's test results window. On the left, a tree view lists the test classes under the package 'example (org)'. The first class, 'OrderServiceTest', is expanded, showing a single test method 'testCreateOrder_whenProductIdIsMissing_throwsOrder' which passed in 1 ms. Below it, 'ProductServiceTest' (14 ms) and 'UserServiceTest' are listed. On the right, a summary bar indicates '3 tests passed' out of 3 tests total, taking 15 ms. Below this, the test output is displayed in a monospaced font, showing the Java library path and three lines of test descriptions: 'Test methods related to User products', 'Test methods related to User', and 'Test methods related to User orders'. At the bottom, it states 'Process finished with exit code 0'.

Test Class	Test Method	Duration
example (org)		15 ms
OrderServiceTest		1 ms
testCreateOrder_whenProductIdIsMissing_throwsOrder		1 ms
ProductServiceTest		14 ms
UserServiceTest		

✓ 3 tests passed 3 tests total, 15 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/

Test methods related to User products

Test methods related to User

Test methods related to User orders

Process finished with exit code 0

Why Control Test Class Order?



Business Logic Dependencies

Useful when business logic depends on sequencing:

- Example: A product cannot be ordered without a user.



Detect Dependencies

Helps detect accidental dependencies between test classes.



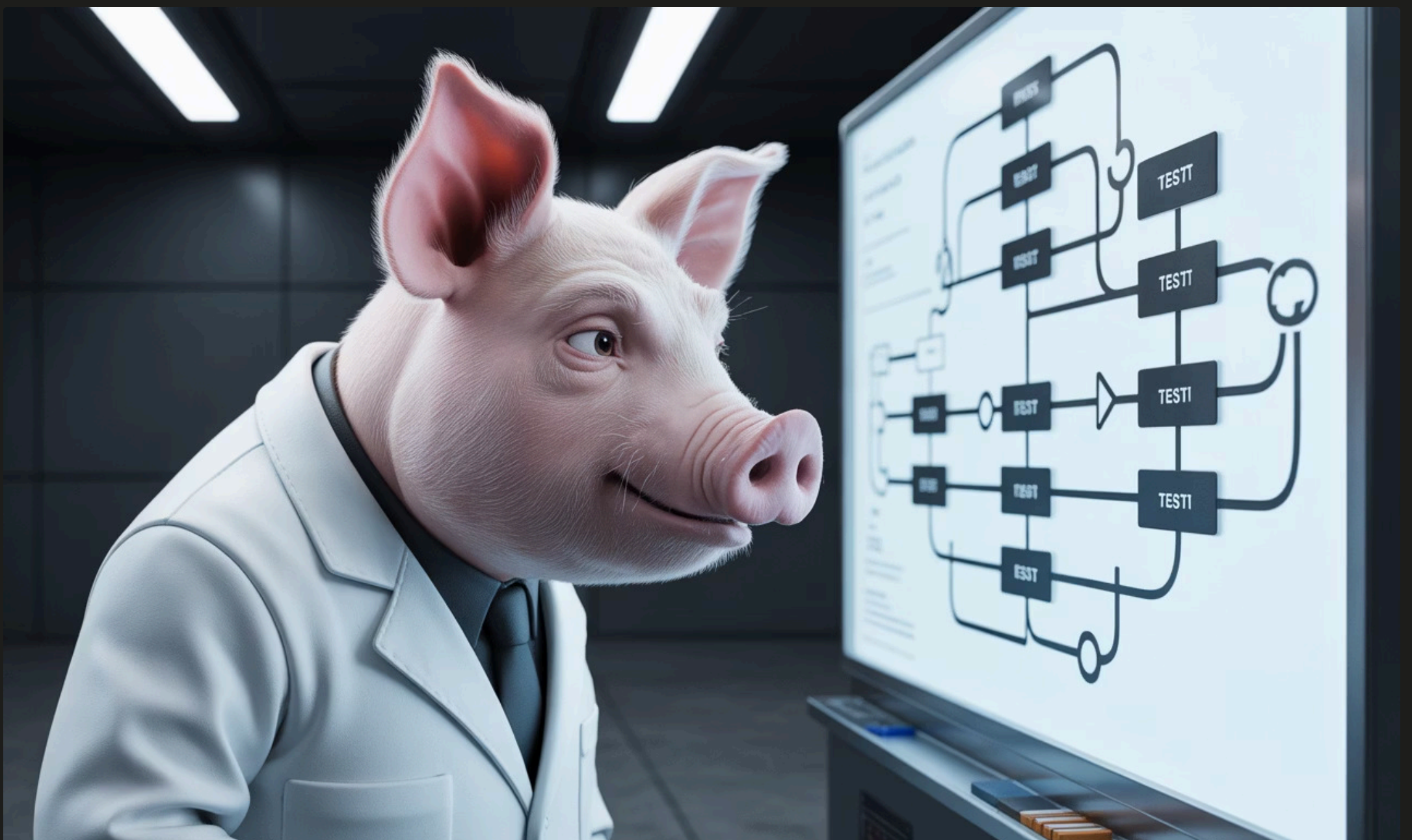
Better Organization

Can improve organization and readability of test reports.



CI Pipeline Control

Allows us to enforce predictable execution in CI pipelines.



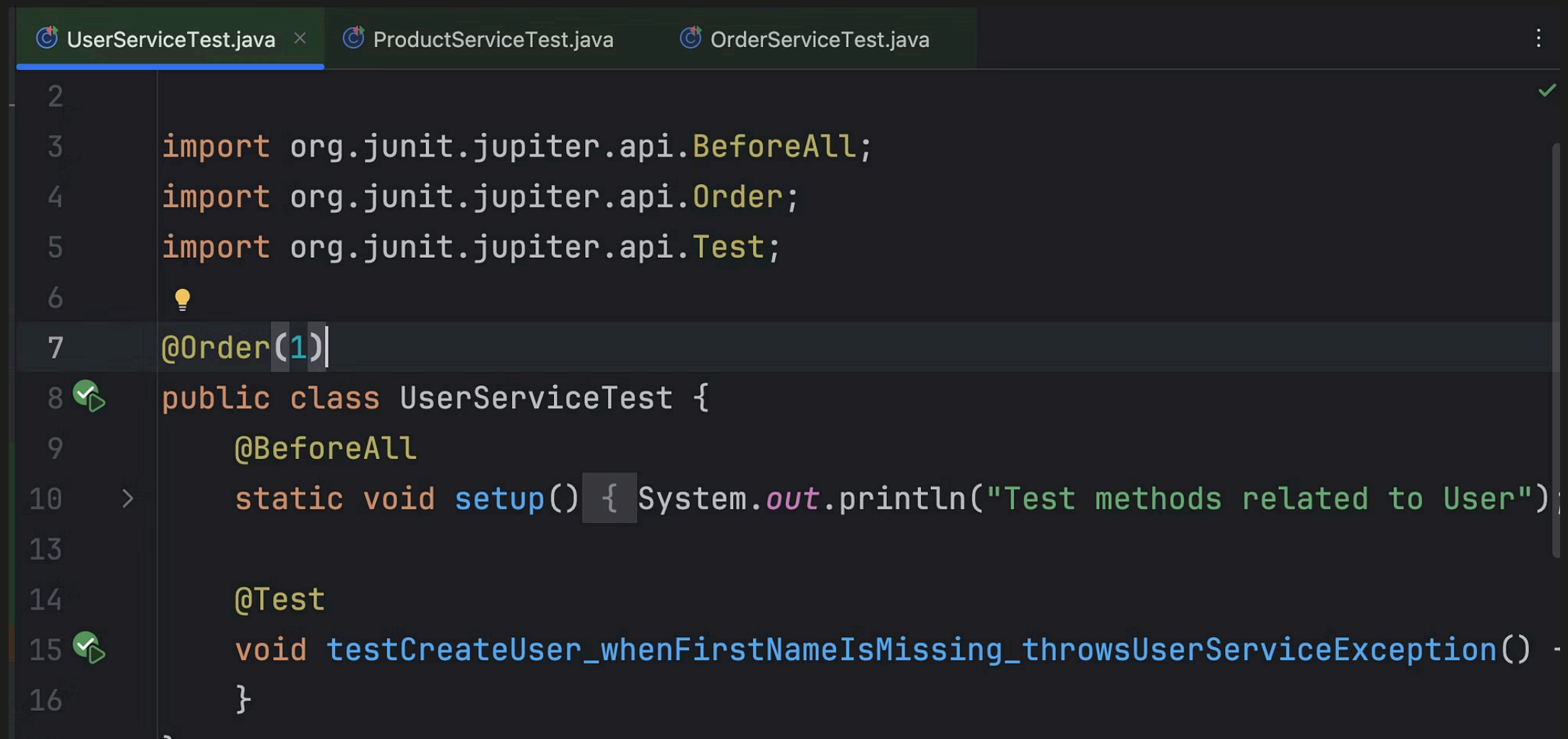
Applying @Order to Test Classes

- **Annotate each class** with `@Order(n)` to define execution sequence.
- Example ordering needed:
 - **1** → `UserServiceTest`
 - **2** → `ProductServiceTest`
 - **3** → `OrderServiceTest`
- Each **class** receives its **own order index**.
- After annotation, JUnit knows which class to run first and last.



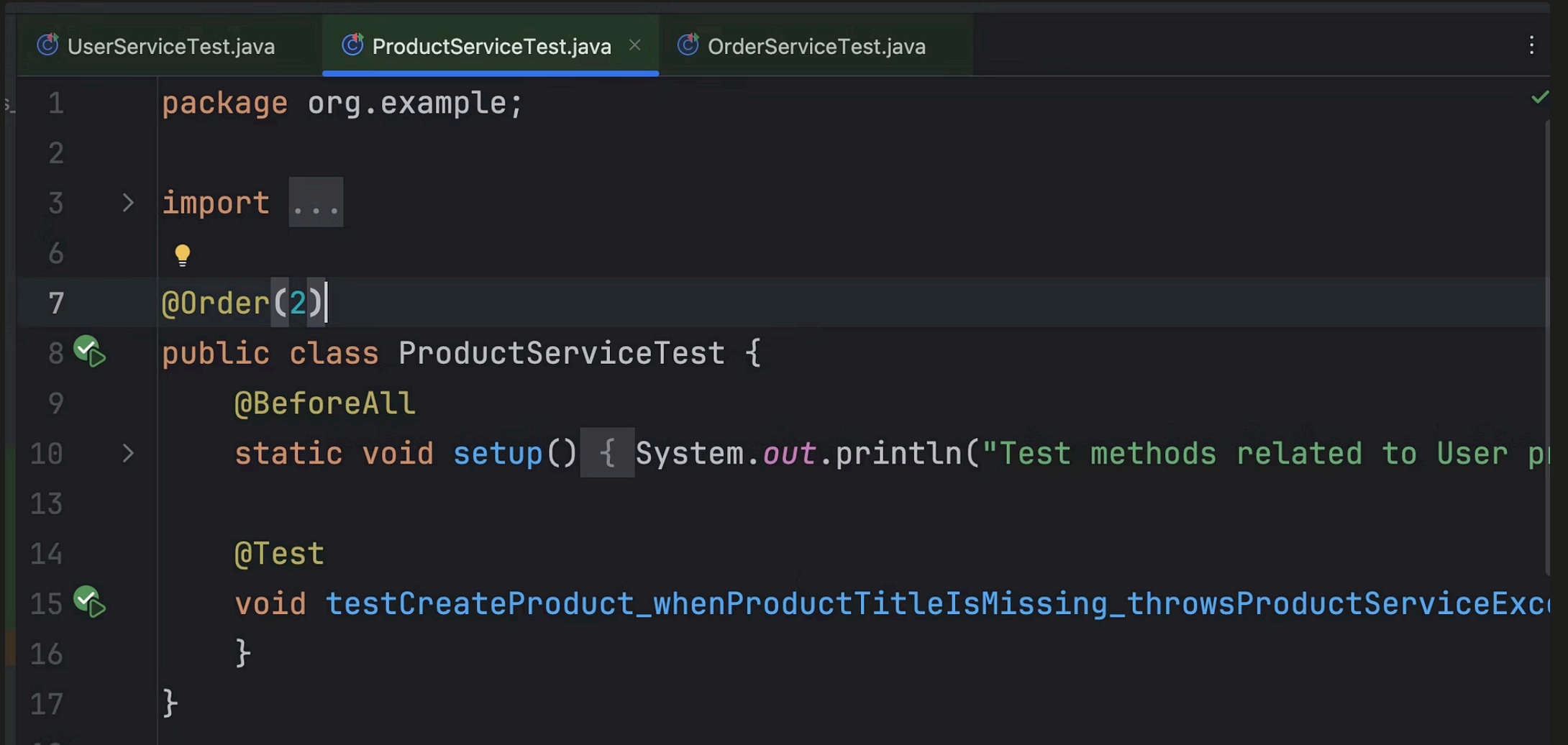
Applying @Order to Test Classes (contd.)

UserServiceTest is the first test class to be executed.



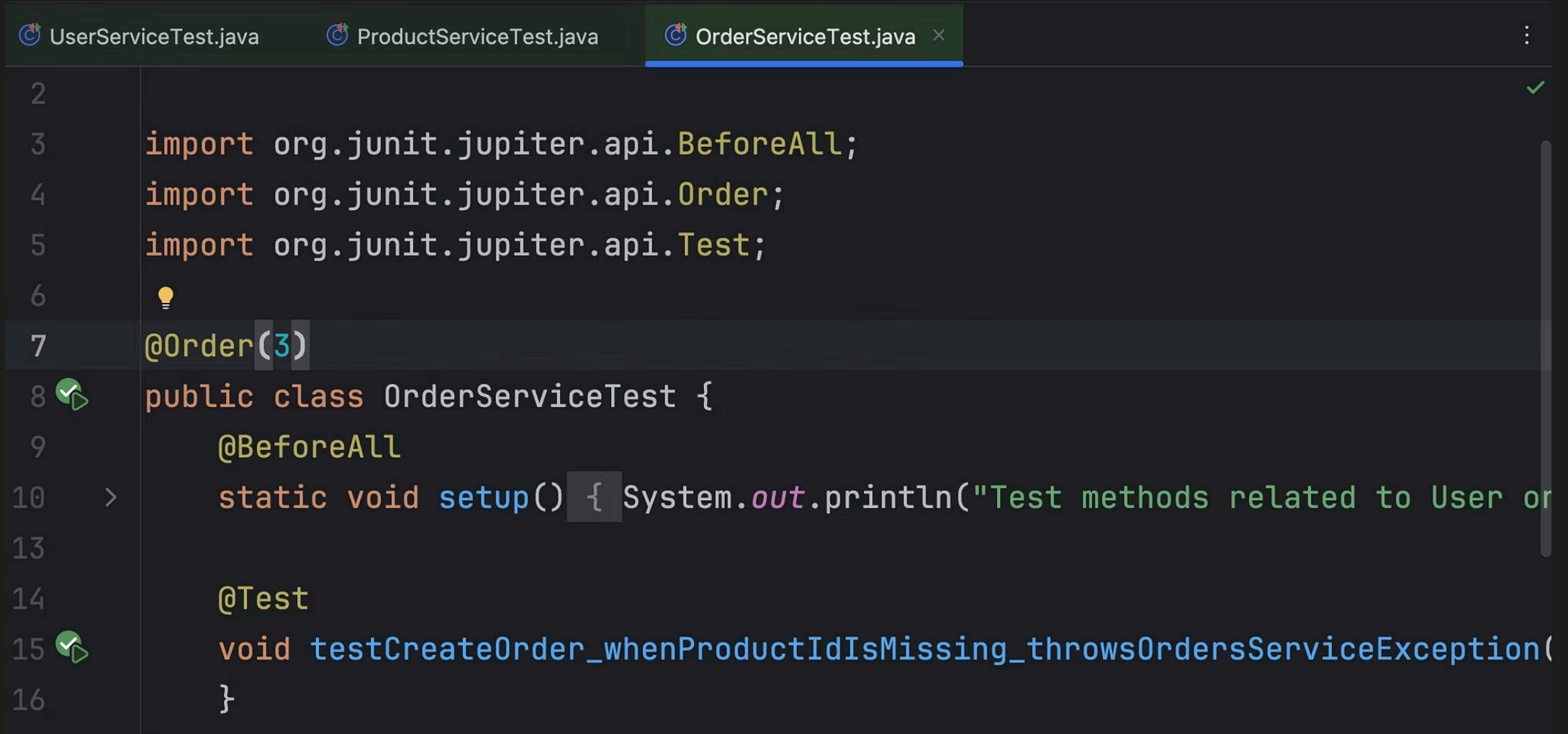
```
UserServiceTest.java × ProductServiceTest.java OrderServiceTest.java
2
3 import org.junit.jupiter.api.BeforeAll;
4 import org.junit.jupiter.api.Order;
5 import org.junit.jupiter.api.Test;
6
7 @Order(1)
8 public class UserServiceTest {
9     @BeforeAll
10     static void setup() { System.out.println("Test methods related to User");
13
14     @Test
15     void testCreateUser_whenFirstNameIsMissing_throwsUserServiceException() {
16
17
```

Applying @Order to Test Classes (contd.)



```
UserServiceTest.java  ProductServiceTest.java x  OrderServiceTest.java  ⋮
1  package org.example;
2
3  > import ...
6  💡
7  @Order(2)
8  ✓▶ public class ProductServiceTest {
9      @BeforeAll
10     > static void setup() { System.out.println("Test methods related to User p
13
14     @Test
15     ✓▶ void testCreateProduct_whenProductTitleIsMissing_throwsProductServiceExc
16     }
17 }
```

Applying @Order to Test Classes (contd.)

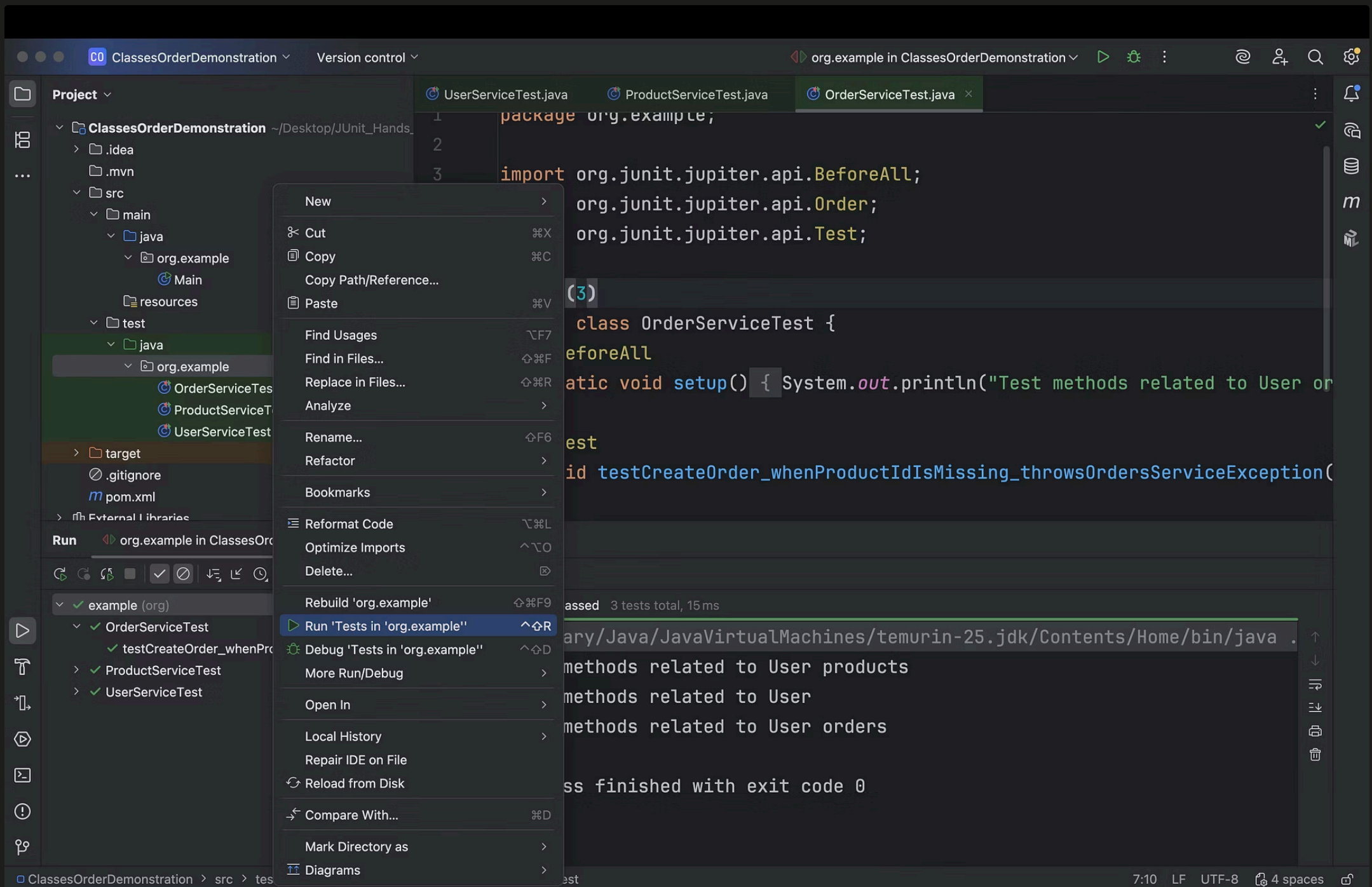


The screenshot shows an IDE with three tabs: UserServiceTest.java, ProductServiceTest.java, and OrderServiceTest.java. The OrderServiceTest.java tab is active. The code in the tab is as follows:

```
2
3 import org.junit.jupiter.api.BeforeAll;
4 import org.junit.jupiter.api.Order;
5 import org.junit.jupiter.api.Test;
6
7 @Order(3)
8 public class OrderServiceTest {
9     @BeforeAll
10     static void setup() { System.out.println("Test methods related to User or");
13
14     @Test
15     void testCreateOrder_whenProductIdIsMissing_throwsOrdersServiceException(
16     }
```

Line numbers 2 through 16 are visible on the left. Line 7, containing the `@Order(3)` annotation, is highlighted. Line 8 has a green checkmark icon. Line 10 has a green arrow icon. Line 15 has a green checkmark icon. A lightbulb icon is present on line 6. A green checkmark is in the top right corner of the editor area.

Running all the test classes again



JUnit ignores the Order annotation

Your tests still run, but **the class execution order you expected is being ignored.**

✓ 3 tests passed 3 tests total, 16 ms

Test methods related to User products

Test methods related to User

Test methods related to User orders

Dec 22, 2025 8:53:24 PM org.junit.platform.launcher.core.DiscoveryIssueNotifier logIssues

INFO: TestEngine with ID 'junit-jupiter' encountered 3 non-critical issues during test discovery:

- (1) [INFO] Ineffective @Order annotation on class 'org.example.ProductServiceTest'. It will not be applied.
Source: ClassSource [className = 'org.example.ProductServiceTest', filePosition = null]
at org.example.ProductServiceTest.<no-method>([SourceFile:0](#))
- (2) [INFO] Ineffective @Order annotation on class 'org.example.UserServiceTest'. It will not be applied.
Source: ClassSource [className = 'org.example.UserServiceTest', filePosition = null]
at org.example.UserServiceTest.<no-method>([SourceFile:0](#))
- (3) [INFO] Ineffective @Order annotation on class 'org.example.OrderServiceTest'. It will not be applied.
Source: ClassSource [className = 'org.example.OrderServiceTest', filePosition = null]
at org.example.OrderServiceTest.<no-method>([SourceFile:0](#))

Process finished with exit code 0

How to avoid JUnit ignoring the Order annotation?

Answer: Configuring Class Order Globally

JUnit allows global configuration via `junit-platform.properties`.

01

Create folder: `test/resources`

02

Mark it as *Test Resources Root* in IDE settings.

03

Create file: `junit-platform.properties`

04

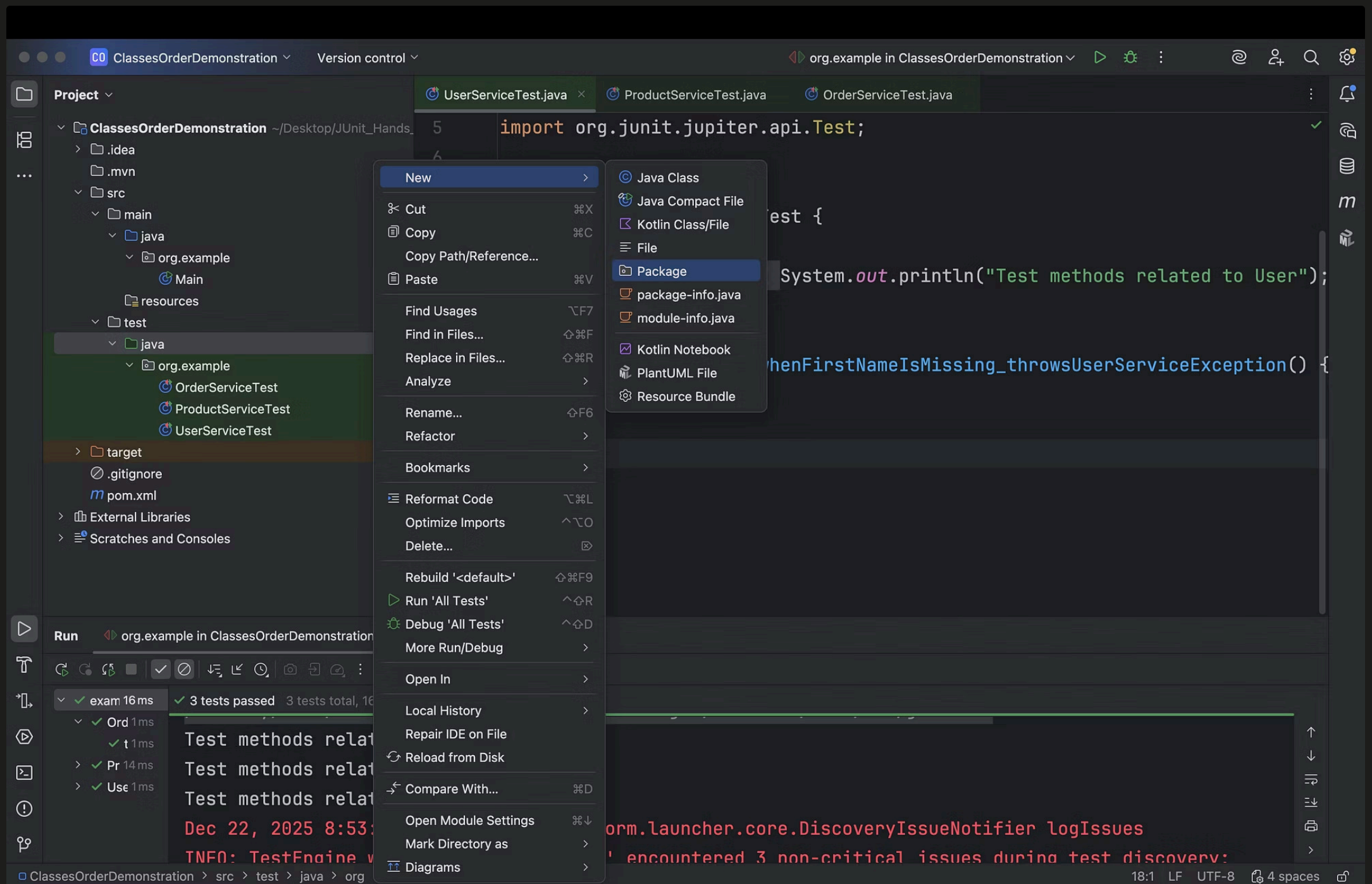
Add `nnnnertu`
`junit.jupiter.testclass.order.default`

```
junit.jupiter.testclass.order.default= org.junit.jupiter.api.ClassOrderer$OrderAnnotation
```

This applies ordering to **all** test classes automatically.

How to avoid JUnit ignoring the Order annotation?

Let's try to order the test classes. First, we will **deliberately** create a `resources` directory in the **wrong place** to demonstrate a typical mistake beginners often make.

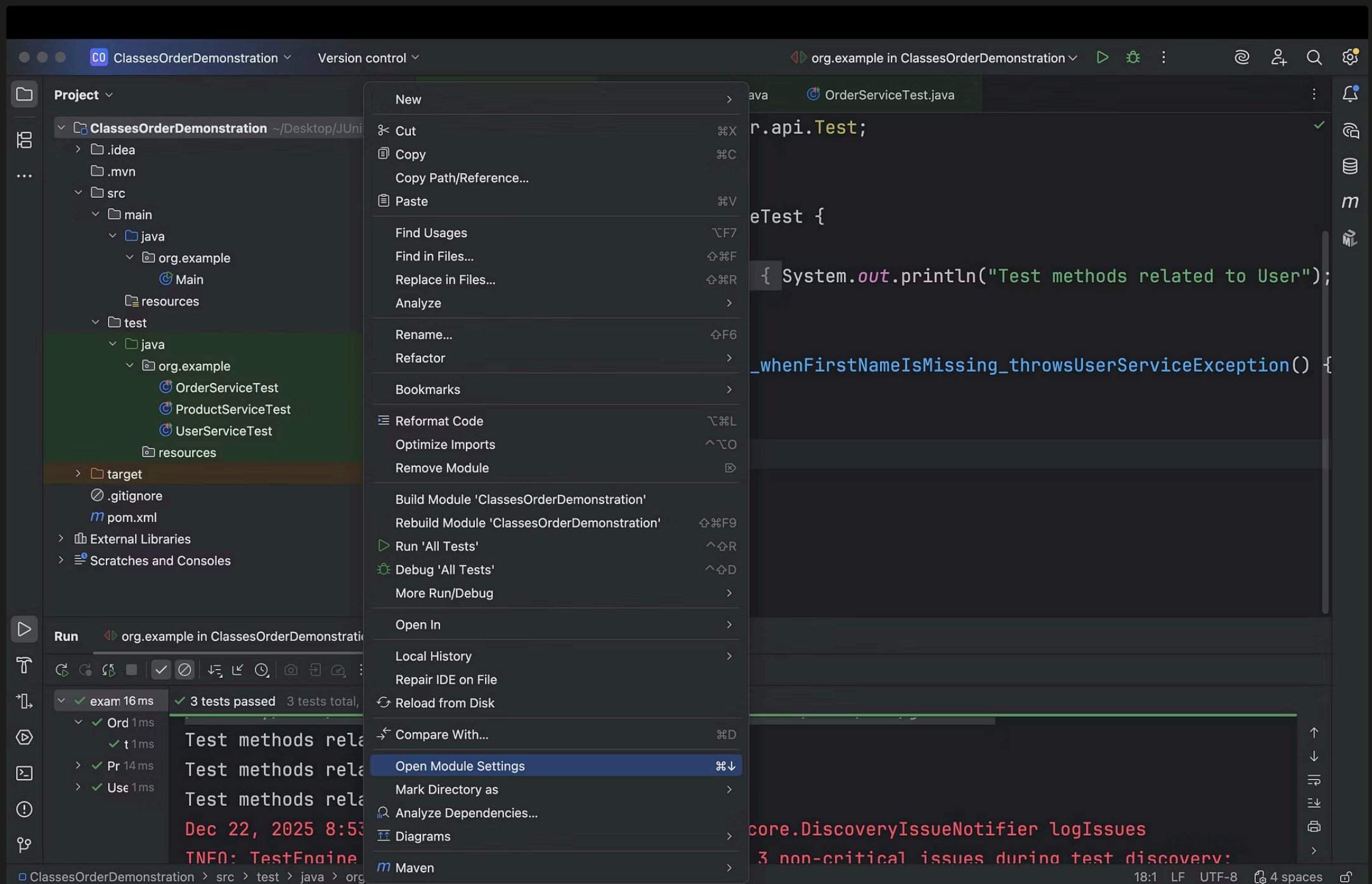


Right-click the `java` folder (wrong choice, since the right choice is the `test` folder) and then select `New` → `Package`

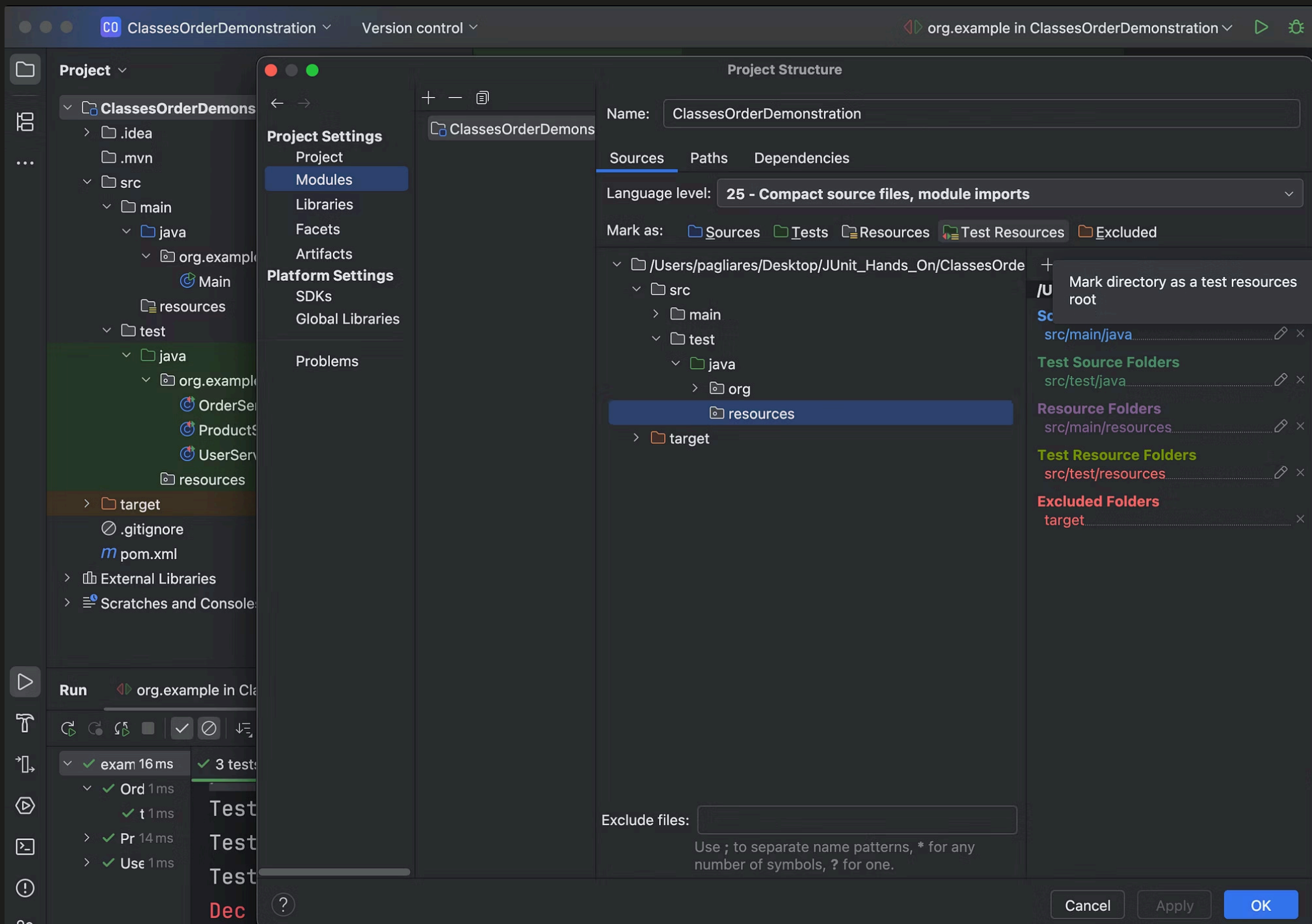
Name the package "`resources`"

Configure the resource folder as a test folder

Right click the project and select "Open Module Settings".



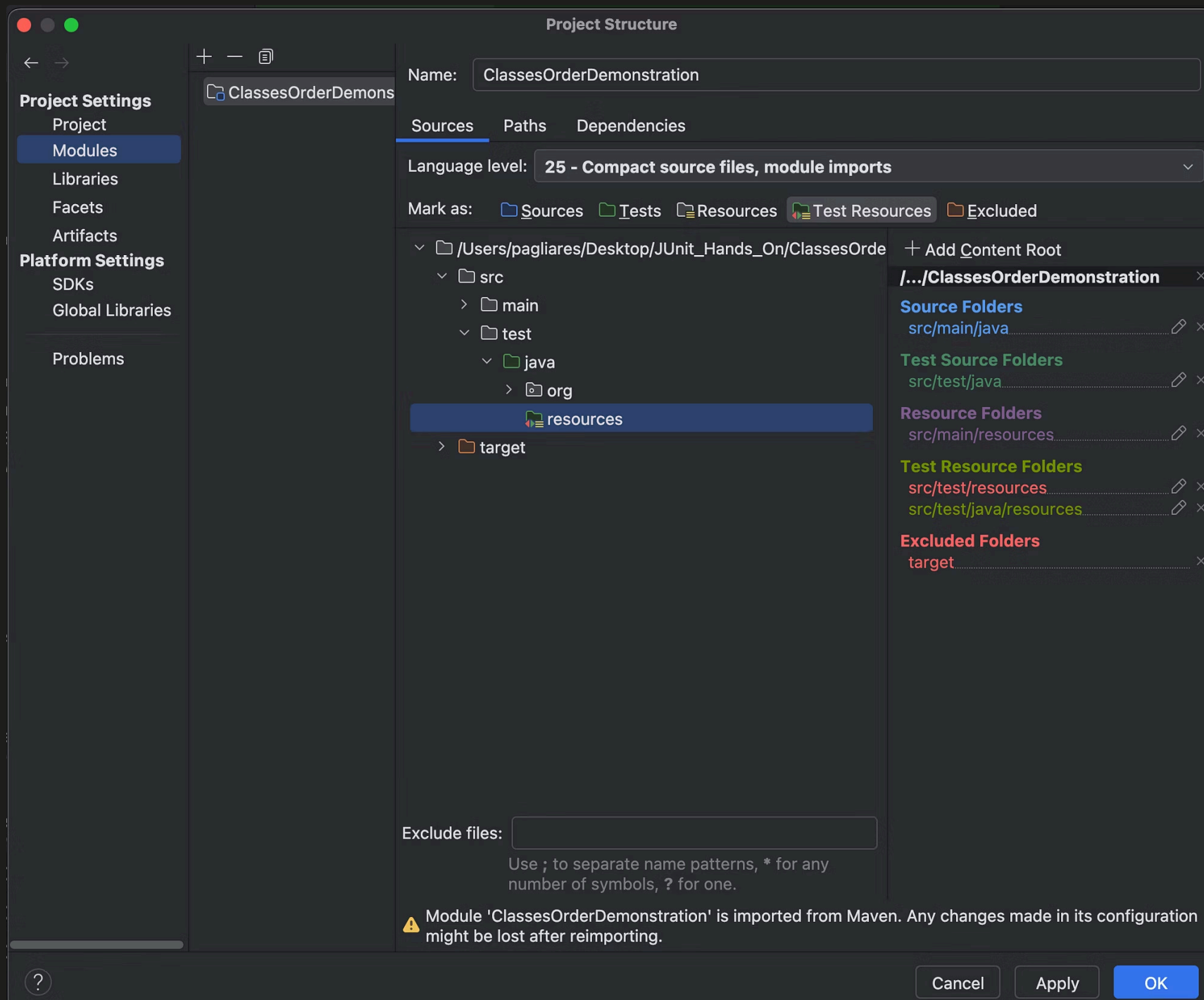
Configure the resource folder as a test folder (contd.)



Select the resources folder and click on the TestResources tab.

Configure the resource folder as a test folder (contd.)

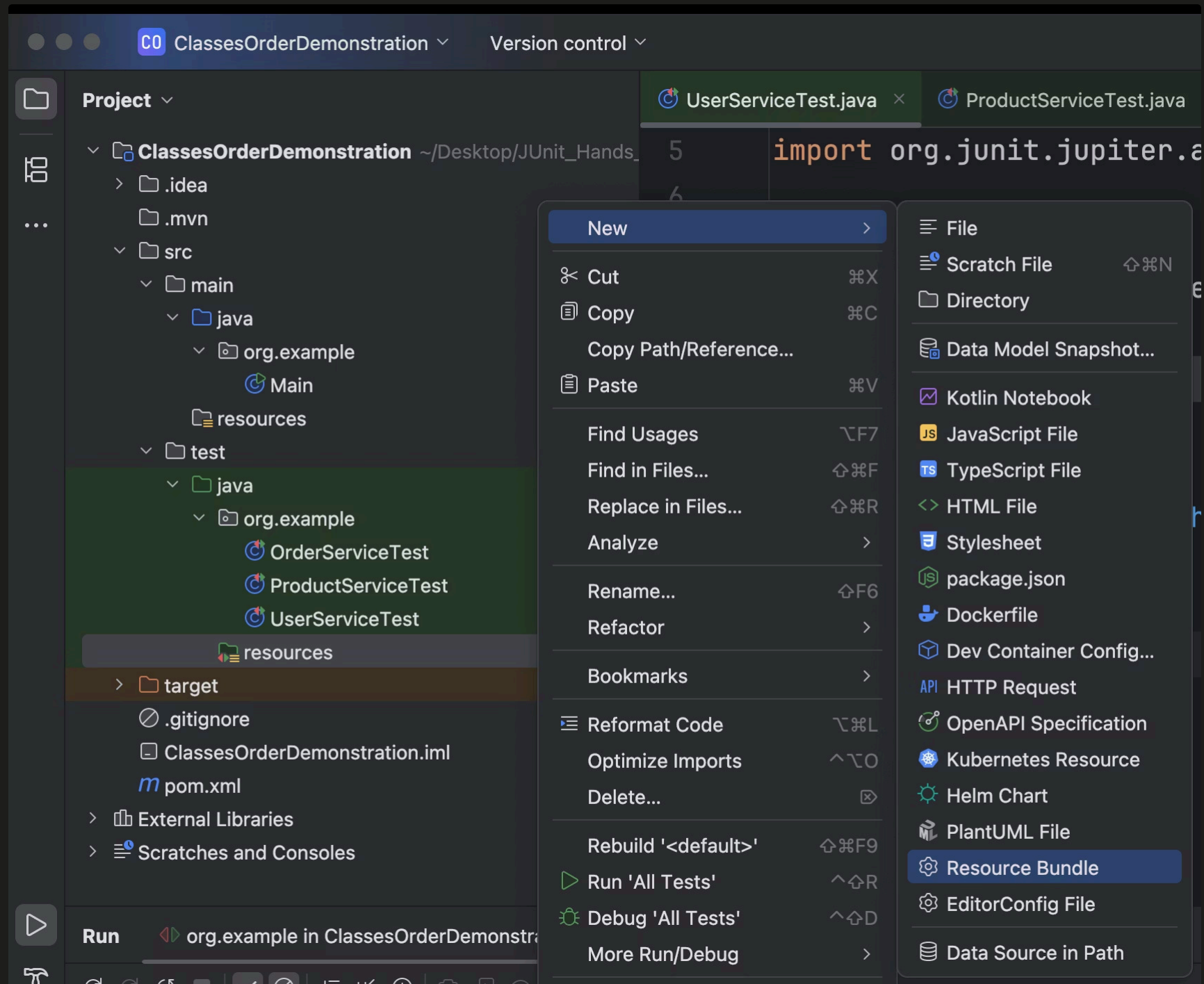
Notice the icon in the resources folder changes.



Click Ok

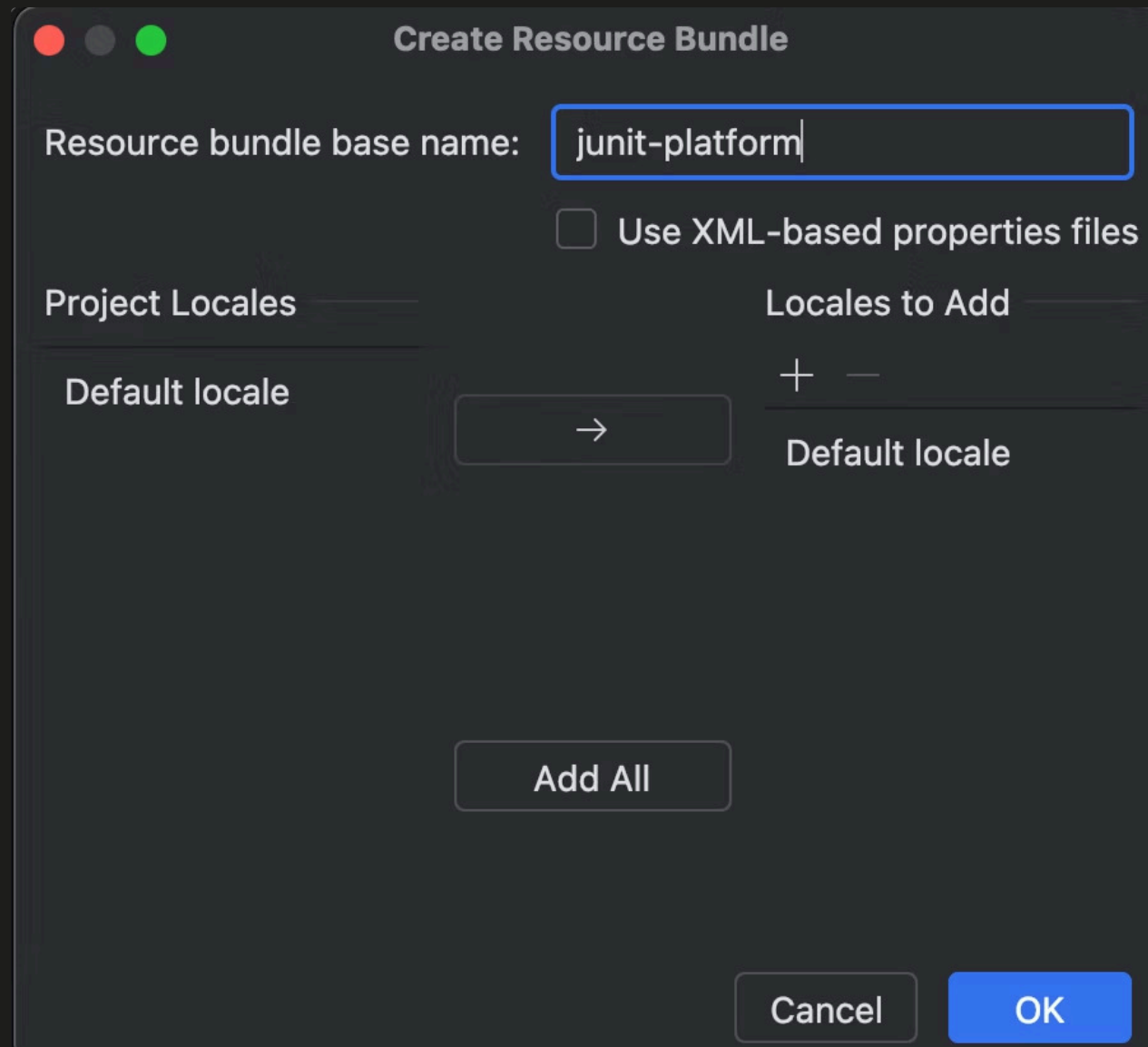
Create a resource bundle

Now we need to create a **resource bundle** to include the configuration that will allow **JUnit** to run the **test classes** in a **specific order**.



Create a resource bundle (contd.)

Name it `junit-platform`



The image shows a 'Create Resource Bundle' dialog box with a dark theme. At the top, the title bar says 'Create Resource Bundle'. Below it, there's a text field for 'Resource bundle base name:' containing the text 'junit-platform'. To the right of this field is a checkbox labeled 'Use XML-based properties files'. Below these, there are two sections: 'Project Locales' on the left and 'Locales to Add' on the right. The 'Project Locales' section has a 'Default locale' label. The 'Locales to Add' section has a '+' and '-' icon and a 'Default locale' label. A right-pointing arrow button is positioned between the two sections. At the bottom center is an 'Add All' button. At the bottom right are 'Cancel' and 'OK' buttons.

Create Resource Bundle

Resource bundle base name:

☐ Use XML-based properties files

Project Locales

Default locale

→

Locales to Add

+ -

Default locale

Add All

Cancel OK

Create a resource bundle (contd.)

Notice we have more than one option to configure the classes execution order. Select `OrderAnnotation` for this lesson.



The screenshot shows an IDE window titled "ClassesOrderDemonstration" with a "Version control" dropdown. The editor displays the file "junit-platform.properties" with the following content:

```
1 junit.jupiter.testclass.order.default=org.junit.jupiter.api.ClassOrderer.OrderAnnotation
2
3
4
5
6
7
```

A dropdown menu is open, showing the following options:

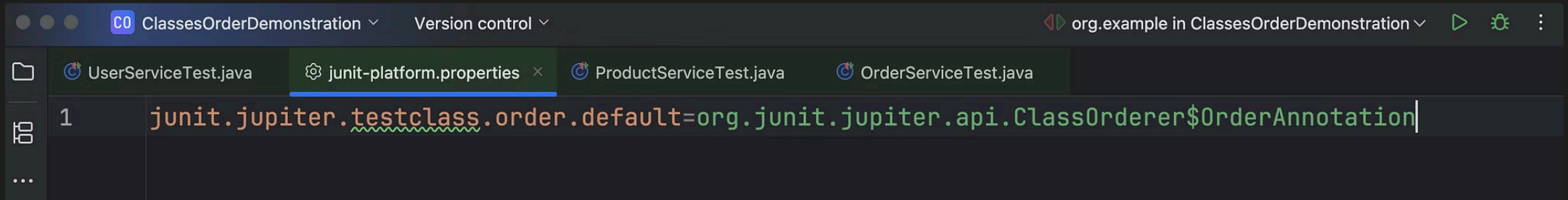
- ClassName
- Default
- DisplayName
- OrderAnnotation
- Random

At the bottom of the dropdown, it says "Press ← to insert, → to replace".

Create a resource bundle (contd.)

IMPORTANT!

You need to use `$` instead of `.` when configuring the the order



The screenshot shows an IDE window titled "ClassesOrderDemonstration" with a "Version control" dropdown. The active tab is "junit-platform.properties". The code editor displays the following line on line 1:

```
junit.jupiter.testclass.order.default=org.junit.jupiter.api.ClassOrderer$OrderAnnotation
```

The IDE interface includes a sidebar with icons for Explorer, Search, and Run and Debug. The top bar shows the current file path: "org.example in ClassesOrderDemonstration".

Create a resource bundle (contd.)

In the same way, we can add a property to **configure test methods order** (not demonstrated in this lesson, hence commented out.)

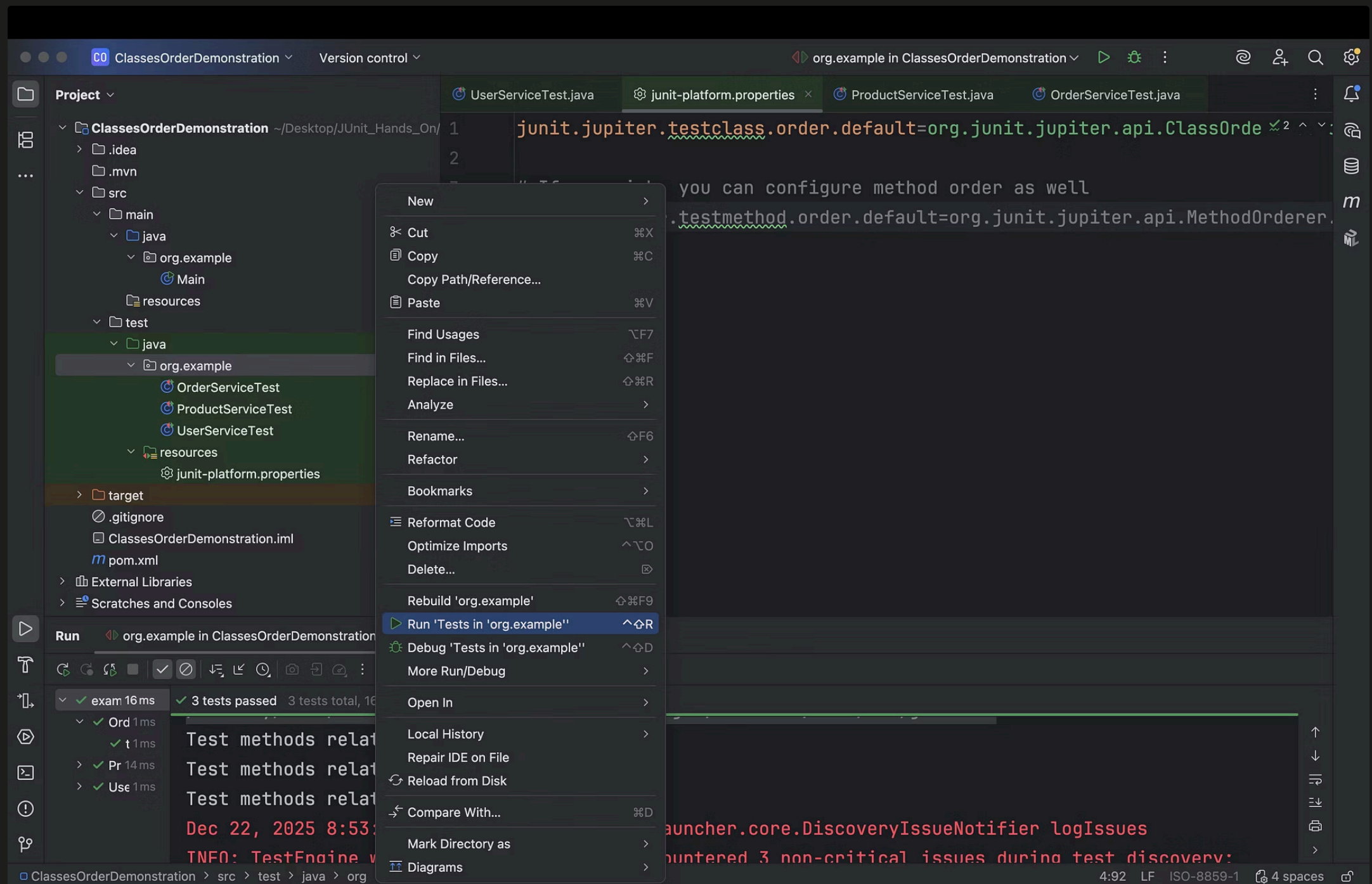


The screenshot shows an IDE window titled 'ClassesOrderDemonstration' with a 'Version control' dropdown. The active tab is 'junit-platform.properties'. The code editor displays the following content:

```
1 junit.jupiter.testclass.order.default=org.junit.jupiter.api.ClassOrderer.OrderAnnotation
2
3 # If you wish, you can configure method order as well
4 #junit.jupiter.testmethod.order.default=org.junit.jupiter.api.MethodOrderer.OrderAnnotation
```

Run the test suite

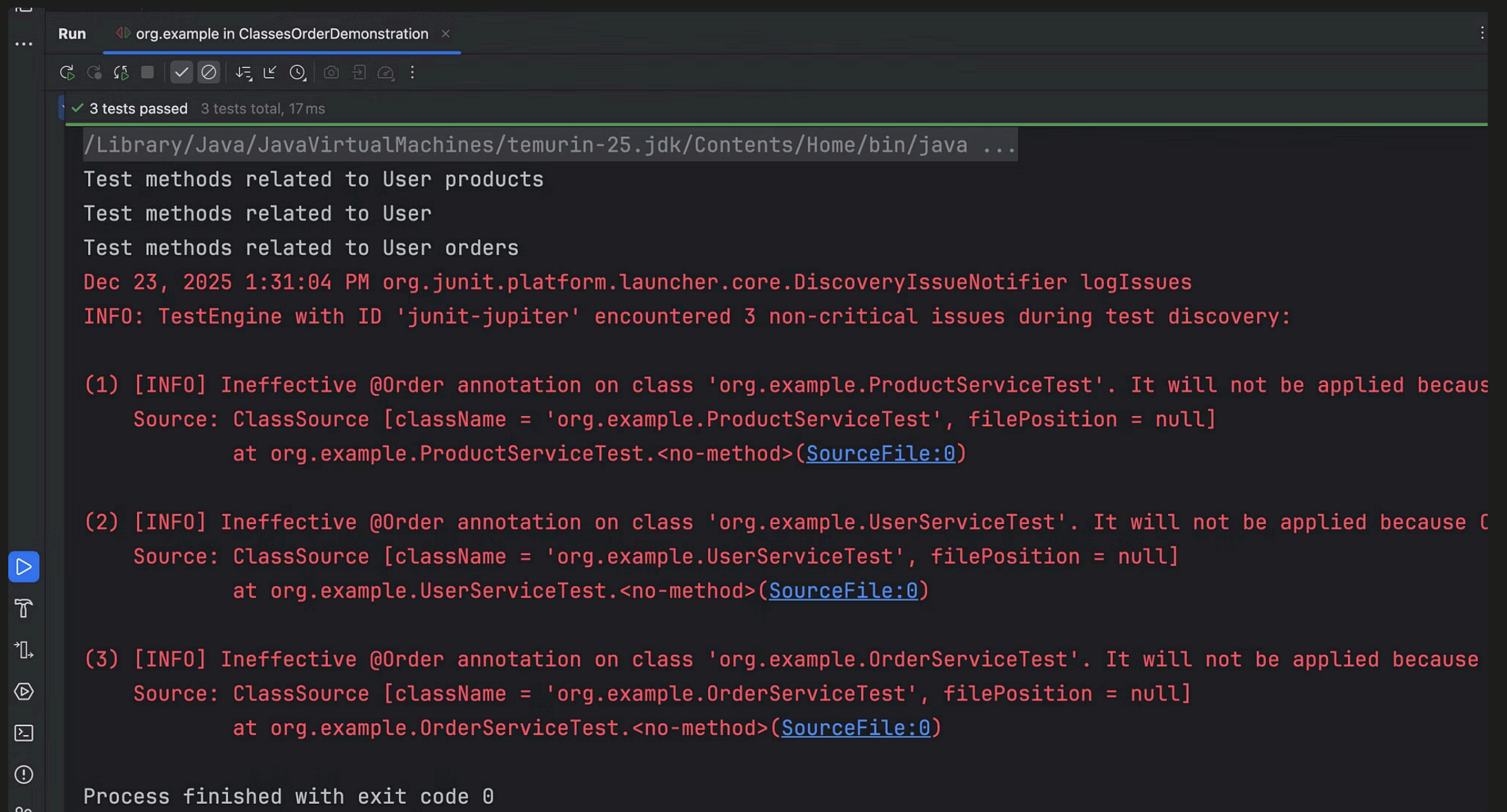
Let's run all **3 test classes** to see whether the **correct order was applied or not**.



Error!

Notice that the `ProductServiceTest` class was executed first, instead of being executed second.

Problem: We created the `resources` folder in the wrong location.



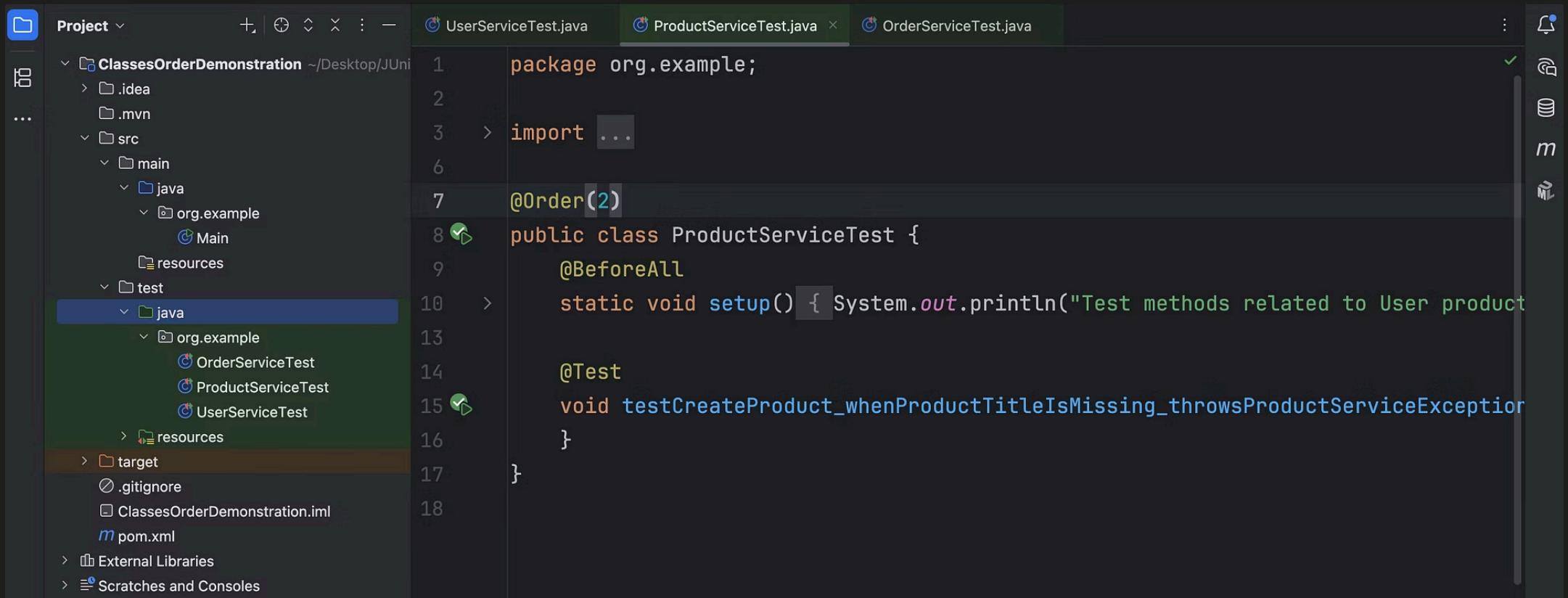
The screenshot shows an IDE's Run console window. At the top, it says "Run" and "org.example in ClassesOrderDemonstration". Below this, a status bar indicates "3 tests passed" and "3 tests total, 17 ms". The main output area shows the following text:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
Test methods related to User products  
Test methods related to User  
Test methods related to User orders  
Dec 23, 2025 1:31:04 PM org.junit.platform.launcher.core.DiscoveryIssueNotifier logIssues  
INFO: TestEngine with ID 'junit-jupiter' encountered 3 non-critical issues during test discovery:  
  
(1) [INFO] Ineffective @Order annotation on class 'org.example.ProductServiceTest'. It will not be applied because  
Source: ClassSource [className = 'org.example.ProductServiceTest', filePosition = null]  
at org.example.ProductServiceTest.<no-method>(SourceFile:0)  
  
(2) [INFO] Ineffective @Order annotation on class 'org.example.UserServiceTest'. It will not be applied because [  
Source: ClassSource [className = 'org.example.UserServiceTest', filePosition = null]  
at org.example.UserServiceTest.<no-method>(SourceFile:0)  
  
(3) [INFO] Ineffective @Order annotation on class 'org.example.OrderServiceTest'. It will not be applied because  
Source: ClassSource [className = 'org.example.OrderServiceTest', filePosition = null]  
at org.example.OrderServiceTest.<no-method>(SourceFile:0)  
  
Process finished with exit code 0
```

The warnings indicate that the `@Order` annotation is ineffective on the `ProductServiceTest`, `UserServiceTest`, and `OrderServiceTest` classes. The console also shows the test methods related to User products, User, and User orders.

Fixing Error!

Copy the resources folder to its right location (under test folder, not java folder).



The screenshot shows an IDE with a project named 'ClassesOrderDemonstration' located at '~\Desktop\JUni'. The project structure in the left sidebar is as follows:

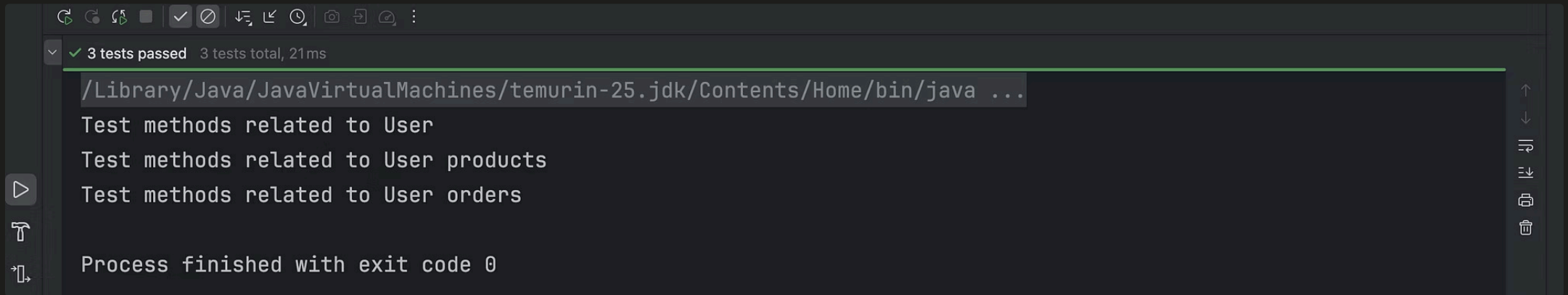
- ClassesOrderDemonstration ~\Desktop\JUni
 - .idea
 - .mvn
 - src
 - main
 - java
 - org.example
 - Main
 - resources
 - test
 - java
 - org.example
 - OrderServiceTest
 - ProductServiceTest
 - UserServiceTest
 - resources
 - target
 - .gitignore
 - ClassesOrderDemonstration.iml
 - pom.xml
 - External Libraries
 - Scratches and Consoles

The main editor displays the code for 'ProductServiceTest.java' (tab 2 of 3). The code is as follows:

```
1 package org.example;
2
3 import ...
4
5
6
7 @Order(2)
8 public class ProductServiceTest {
9     @BeforeAll
10    static void setup() { System.out.println("Test methods related to User product"); }
11
12
13
14    @Test
15    void testCreateProduct_whenProductTitleIsMissing_throwsProductServiceException() {
16    }
17 }
18
```

Run the test suite again!

Let's run all **3 test classes** to see whether the **correct order was applied or not**.

A screenshot of an IDE terminal window. The top status bar shows a green checkmark and the text "3 tests passed 3 tests total, 21ms". The terminal output shows the command path "/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ..." followed by three lines of test results: "Test methods related to User", "Test methods related to User products", and "Test methods related to User orders". At the bottom, it says "Process finished with exit code 0". The terminal has a dark theme and various icons on the left and right sides.

```
✓ 3 tests passed 3 tests total, 21ms  
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
Test methods related to User  
Test methods related to User products  
Test methods related to User orders  
  
Process finished with exit code 0
```

Tests passed!

Alternatives for Global Ordering

Other built-in class order strategies:

ClassOrderer\$Random

Run classes in random order.

ClassOrderer\$ClassName

Alphabetical order.

Similar configuration possible for **method** ordering.

Use wisely: tests should not rely on order unless necessary.



Conclusion

- Use **@Order** when class execution must follow a specific sequence.
- Configure **junit-platform.properties** for project-wide behavior.
- Running all tests now produces:
 - 1st: User
 - 2nd: Product
 - 3rd: Order
- Method ordering can be globally configured in the same way.

Open Floor Discussion

- When is class ordering truly necessary vs. an **anti-pattern**?
- How does **ordered execution affect test independence**?
- Should **integration tests** be separated from **unit tests** to avoid ordering concerns?